Physics Is All You Need: Lessons from Physicist-Supervised AI Development of Scientific Software

Anonymous Authors[1]

Abstract

Are AI agents tools, co-authors, or autonomous scientists? We present a quantified case study: a physicist supervising an AI coding agent (Claude Code, Sonnet and Opus models) over 12 days and 57 sessions to build a differentiable one-loop perturbation theory (a next-to-leading-order calculation for predicting galaxy clustering) module in JAX ($\sim$2,100 lines, validated to $\lesssim 1\%$ accuracy against CLASS-PT). We documented 15 issues and classified each by whether autonomous resolution was possible.

The agent resolved ten autonomously (convention errors, algorithm transcription, numerical coefficients) by iterating against oracle test suites. The other two were accelerated by the physicist's domain knowledge. The three it could not—all evaded oracle detection—share a common property: the agent treated symptom reduction as equivalent to root-cause resolution. It spent 33 of the 57 sessions tuning coefficients within a fundamentally wrong code architecture, unable to recognize the problem was structural rather than parametric. After the physicist directed a redesign, the agent then found a scalar correction by grid search that reduced error below 1% across all test cases—but the value corresponds to no physical quantity and would silently produce wrong predictions at any other cosmology.

Three supervision practices, developed iteratively, proved critical for catching what oracle tests missed: testing against diverse cosmologies beyond the fiducial calibration; shared changelogs that surfaced stalled exploration across sessions; and an explicit rule against unphysical numerical patches. In our case study, the design of these supervision protocols (not model capability) was the primary factor in whether the agent's output was trustworthy. Closing this gap would require agents that can propose architectural alternatives rather than optimizing within a given structure, and distinguish predictive adequacy from explanatory correctness—capabilities not yet exhibited by current LLMs and not obviously addressed by scaling alone.

1. Introduction

Most empirical evidence on AI coding agents in science comes from one of two regimes: standardized coding benchmarks and large-scale software projects where automated testing fully captures correctness (Carlini, 2026), or fully autonomous multi-agent systems that generate manuscripts without sustained human oversight (Villaescusa-Navarro et al., 2025). Neither captures the reality of scientific software development, where correctness is defined by agreement with physical law rather than by compilation or test passage, and where physicists work with AI agents over weeks to produce validated code.

We present a detailed case study of exactly this scenario. Over 12 days and 57 agent sessions, a physicist supervised an AI coding agent (Claude Code, powered by Anthropic's Sonnet and Opus models) to build CLAX-PT, a differentiable one-loop perturbation theory module (computing next-to-leading-order predictions for galaxy clustering) in JAX. The module computes nine output power spectra validated to $\lesssim 1\%$ accuracy against the established reference code CLASS-PT (Chudaykin et al., 2021). The resulting implementation comprises approximately 2,100 lines of code. The contribution of this paper is not the code itself (described in a companion paper) but the empirical supervision record: what the human did, what the agent did, and where the boundary between their roles became load-bearing.

Our central claim is that the supervision protocol—not the model's capability—was the primary factor determining whether the agent's output was trustworthy scientific software. The agent autonomously resolved 10 of the 15 documented issues by iterating against oracle test suites. Two

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the AI4Physics Workshop at the International Conference on Machine Learning (AI4Physics@ICML 2026). Do not distribute.

more were accelerated by human domain knowledge. The three it could not resolve were invisible to oracle testing and required human physics judgment to diagnose. These three bugs consumed a disproportionate fraction of the project: 33 of 57 sessions were spent within a fundamentally wrong code architecture that the agent could not recognize as wrong because it passed local consistency checks. The physicist's interventions—an architectural redesign and the rejection of a physically unmotivated numerical patch—were the decisive acts that produced correct code. Both share a common cognitive structure: the physicist evaluated not whether the code produced right numbers, but whether it produced them *for the right reasons*.

2. The Project

2.1. What Was Built

We compute predictions for the gravitational clustering of galaxies, the three-dimensional distribution mapped by galaxy redshift surveys. The calculation involves loop integrals (analogous to next-to-leading-order corrections in quantum field theory) that are expensive to evaluate and sensitive to subtle physical effects: bulk motions of matter and oscillation features imprinted by sound waves in the early universe. Getting these effects wrong by even 1% can bias the cosmological parameters (the parameters of the standard model of cosmology) inferred from the galaxy power spectrum.

One-loop perturbation theory (the next-to-leading-order correction beyond the linear approximation) extends the linear matter power spectrum to mildly nonlinear scales by computing correction terms from second- and third-order density fields, producing predictions for galaxy clustering that are essential for extracting cosmological parameters from spectroscopic surveys (D'Amico et al., 2020; Perko et al., 2016). CLAX-PT implements this calculation in JAX: it takes a linear power spectrum and cosmological parameters as input, evaluates the tree-level and one-loop terms (the latter via FFTLog decomposition (Simonović et al., 2018; Fang et al., 2017)), applies infrared (IR) resummation (Senatore & Zaldarriaga, 2015; Vlah et al., 2015), and adds ultraviolet (UV) counterterms that absorb sensitivity to small-scale physics outside the perturbative regime (Baumann et al., 2012; Carroll et al., 2014). IR resummation corrects for large-scale bulk flows which, if unaccounted for, artificially smear the baryon acoustic oscillation (BAO) feature, the standard-ruler imprint of primordial sound waves. The code returns nine validated output spectra (Table 1). The implementation is approximately 2,100 lines of JAX code, end-to-end differentiable, and validated to $\lesssim 1\%$ accuracy against CLASS-PT (Chudaykin et al., 2021). Technical details are described in a companion paper.

Table 1. Accuracy of CLAX-PT vs. CLASS-PT at Planck 2018 fiducial cosmology, $z = 0.38$, $k < 0.3$ h/Mpc. Hexadecapole uses $|\Delta|/\max(|\mathrm{ref}|)$ due to zero crossings.

Spectrum	Max error	Mean error
P_{mm}, P_{gg}, P_{gm} (real-space)	0.31%	0.04%
Monopole ($\ell = 0$)	0.59%	0.40%
Quadrupole ($\ell = 2$)	0.89%	0.50%
Hexadecapole ($\ell = 4$)	1.43%	0.37%

2.2. The Supervision Protocol

Before any code was written, the physicist established a supervision protocol adapted from the methodology of Anthropic's C-compiler agent project (Carlini, 2026), which used 16 parallel agents over 2,000 sessions to build a compiler that successfully builds the Linux kernel. Four infrastructure elements structured every session:

1. **CLASS-PT as oracle.** Every function was tested against reference data generated by the established Fortran implementation. Tests were written before code—the agent knew what the correct output looked like before attempting to produce it.

2. **CHANGELOG as shared memory.** Because each agent session starts with no memory of previous sessions, a structured log recorded what was attempted, what failed, and what succeeded. This prevented later sessions from re-exploring dead ends (e.g., a discrete sine transform (DST) grid bug that was solved on day 3 was never re-investigated).

3. **A `--fast` flag for context-window hygiene.** Tests printed at most 10 lines on success and 20 on failure. Verbose diagnostics were written to log files. This discipline (drawn directly from Carlini (2026)) prevented the agent's finite context window from being consumed by noise.

4. **Parallel agent sessions via git worktrees.** Multiple sessions explored competing hypotheses simultaneously. When a bug had multiple plausible causes, the physicist spawned parallel investigations rather than serializing them.

Two rules proved critical beyond this infrastructure. First, "no fudge factors": if a test failed at 0.2% error, there was a real bug to find—a multiplicative correction tuned to make the test pass would be rejected at review. Second, "test at multiple parameter points": the oracle compared not just at the fiducial Planck 2018 cosmology but at varied cosmological parameters, to prevent solutions that were calibrated to a single point. The supervision protocol—not individual code edits—was the physicist's primary contribution to the project.

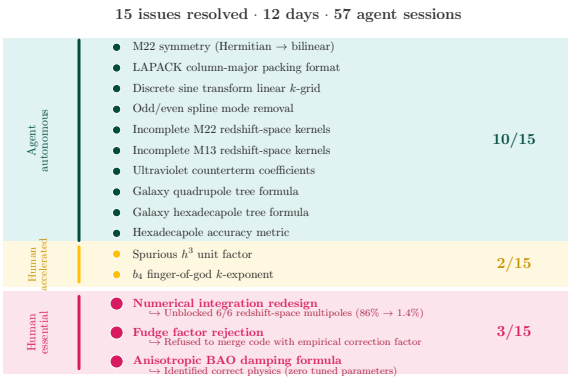

Figure 1. Bug taxonomy for the CLAX-PT development. Ten issues were resolved autonomously by the agent iterating against oracle tests. Two were accelerated by the physicist's domain knowledge (unit bugs invisible to shape-based comparisons). Three required essential human physics judgment: an architectural redesign, a fudge-factor rejection, and identification of the correct physical formula.

3. Bug Taxonomy: The Autonomy Spectrum

We documented 15 issues over 57 agent sessions and classified each by whether autonomous resolution was possible (Figure 1). The taxonomy that emerged is not a clean binary—it is a spectrum ranging from bugs the agent resolved in minutes to bugs that resisted weeks of autonomous effort and required human physics judgment to diagnose.

3.1. Autonomous and Human-Accelerated Issues (12 of 15)

The agent resolved 10 bugs without human intervention by iterating against the oracle test suite. Two more (the h^3 unit factor and b_4 k-exponent) were accelerated by the physicist spotting magnitude discrepancies invisible to shape-based comparisons. These fell into three categories. First, convention and unit errors: the FFTLog matrix (used to decompose loop integrals) M_{22} required Hermitian rather than symmetric packing (bug 1), LAPACK column-major ordering differed from the row-major layout the agent initially produced (bug 2), and a spurious h^3 factor multiplied all bias functions (bug 5). Second, algorithm implementation errors: a DST grid used linear rather than logarithmic spacing (bug 3), odd/even spline interpolation for separating the BAO oscillation feature used the wrong parity assignment (bug 4), and three redshift-space distortion (RSD) kernel matrices were incomplete (bugs 7–9). Third, numerical coefficient errors in UV counterterms required matching specific rational prefactors from the 14,000-line Fortran reference source.

In all these cases, the pattern was the same: the oracle test produced a clear numerical discrepancy, the agent compared intermediate quantities against reference data to localize the error, and the fix was a direct transcription correction.

3.2. Case Study: The Accuracy Wall

After the first 24 sessions resolved bugs 1–9, all three real-space power spectra passed at sub-percent accuracy. The six RSD multipoles (monopole, quadrupole, and hexadecapole for both matter and galaxies) did not. Errors ranged from 8% to 86% depending on the multipole and varied erratically as the agent adjusted individual terms (Figure 2).

Over the next 33 sessions (sessions 24–56 of 57 total), the agent attempted to fix the RSD multipoles within the existing code architecture. This architecture computed analytic Legendre projections of the one-loop integrands: for each multipole ℓ and each component (velocity-velocity, velocity-density, density-density), a pre-derived kernel matrix encoded the angular projection. The approach is correct when the infrared resummation factor (the exponential damping at BAO scales) is isotropic, meaning it depends only on wavenumber k and not on the angle μ between the wavevector and the line of sight.

Over these 33 sessions, the agent worked coherently within this architecture: adjusting kernel coefficients, adding angular terms, swapping quadrature rules, and comparing matrix elements against the reference source term by term. Each fix improved one multipole while degrading another. The error surface was non-convex within this model: no combination of coefficient adjustments could make all six multipoles pass simultaneously, because the architecture itself was structurally incompatible with the physics.

The physicist diagnosed the structural issue by recognizing what the agent could not: the BAO damping in redshift space is *anisotropic*. Peculiar velocities along the line of sight enhance the displacement field. The damping factor depends on the angle μ between the wavevector and the line of sight: $\Sigma^2_{\rm tot}(\mu) = (1 + f(2 + f)\mu^2)\sigma_v^2 + f^2\mu^2(\mu^2 - 1)\sigma^2_{\rm BAO}$, where f is the linear growth rate of structure (how fast density perturbations amplify over time). This μ-dependent exponential cannot be absorbed into a polynomial kernel matrix—it couples to the RSD Kaiser factor $(1 + f\mu^2)^2$ (the leading-order enhancement of clustering along the line of sight due to galaxy peculiar velocities) in a way that no finite set of pre-computed analytic projections can represent exactly.

The physicist proposed a redesign: abandon per-multipole analytic kernels entirely, assemble the full anisotropic power spectrum $P(k, \mu)$ at each of N Gauss–Legendre quadrature nodes in μ, and integrate numerically against Legendre polynomials to extract multipoles. The agent implemented this redesign in a single session. Errors for all six RSD multipoles dropped from 8–86% to the 1–2% range, with four of

six passing sub-percent immediately. The remaining two quadrupoles required an additional correction (Section 3.3).

Why could the agent not find this itself? Two factors conspired. First, CLASS-PT has two parallel code paths for these integrals. The agent consulted only the simpler one, which uses isotropic damping and analytic projections—exactly the architecture the agent had implemented. The correct anisotropic formula lives in the other path, which handles geometric distortion corrections that the test configuration did not require. Second, recognizing that the architecture is wrong (rather than merely mis-parameterized) requires understanding the physics of anisotropic BAO damping: specifically, that the μ-dependence of $\Sigma^2_{\rm tot}$ makes the exponential non-polynomial in μ. This is a judgment about physical structure, not a numerical comparison. Of these two factors, the first is an engineering limitation (incomplete codebase search) addressable by retrieval tools. The second is the explanatory agency gap that no amount of search can close.

3.3. Case Study: The Fudge Factor

After the Gauss–Legendre redesign, seven of nine spectra passed. Two quadrupole spectra ($P_{mm,\ell=2}$ and $P_{gg,\ell=2}$) failed at scales near the BAO peak (the primordial sound-wave imprint) with errors of 1.4% and 1.7% respectively. The agent traced the discrepancy to the tree-level (leading-order, before loop corrections) power spectrum, which uses the resummed form $P_{\rm tree}(k) = P_{\rm nw}(k) + e^{-\Sigma^2 k^2}(1 + \Sigma^2 k^2)\, P_{\rm w}(k)$, where $P_{\rm nw}$ and $P_{\rm w}$ are the no-wiggle and wiggle components, and Σ^2 is the BAO damping scale.

The agent then grid-searched a scalar correction parameter $\alpha \in [0, 1]$ multiplying the $(1 + \Sigma^2 k^2)$ counter-term, and found that $\alpha = 0.27$ minimized the worst-case error to below 1% across all nine spectra simultaneously. The agent committed this fix with a passing test suite. The "no fudge factors" rule had been in CLAUDE.md from the start of the project, two months earlier. The agent did not flag $\alpha = 0.27$ as a violation, framing α as a free parameter inside an existing physics formula rather than as a tuned correction. The literal rule was followed; the principle was missed.

This solution is wrong, despite passing every oracle test. The value $\alpha = 0.27$ has no physical derivation. It is not a parameter in CLASS-PT. It is not derived from any perturbation theory calculation—unlike the effective field theory (EFT) counterterms (c_0, c_2, $\tilde{c}$), which are also free parameters but have clear theoretical motivation as absorbers of UV sensitivity within a controlled expansion. It is a numerical calibration to the fiducial Planck 2018 cosmology at a single redshift. Changing the baryon density ω_b, the cold dark matter density ω_{cdm}, or the effective redshift would shift the BAO peak amplitude, and $\alpha = 0.27$ would produce predictions wrong by an amount invisible to the existing test suite, because the test suite only checks the fiducial cosmology.

The physicist diagnosed the fudge factor by asking a question the agent could not formulate: "what does α correspond to in the perturbation theory derivation, or did you just copy it from the CLASS-PT source?" The agent confirmed neither: α appears nowhere in the reference, and no derivation produces it. The physicist then pointed out that the tree-level spectrum $P_{\rm tree}$ (the building block of the one-loop calculation) should carry the same anisotropic damping $\Sigma^2_{\rm tot}(\mu)$ around the BAO scale that already appeared in the Gauss–Legendre loop, rather than the isotropic form the agent had inherited from the simpler CLASS-PT branch. The fix was to move the tree-level computation inside the existing quadrature loop—three lines of code, replacing the committed fudge factor with the correct anisotropic formula. After this fix, all nine spectra passed with zero tuned parameters.

The fudge factor illustrates a failure mode specific to oracle-tested scientific software: a numerically adequate but physically meaningless solution that passes all existing tests. The agent cannot distinguish between "this works because it is correct" and "this works because it is calibrated to the test data." The physicist could, because the physicist knew that $\alpha = 0.27$ must correspond to *something* in the theory—and when it corresponded to nothing, the solution was suspect regardless of its numerical performance.

The three human-essential interventions were thus: (i) the Gauss–Legendre architectural redesign that unblocked all six redshift-space multipoles (Section 3.2), (ii) the rejection of the fudge factor $\alpha = 0.27$ as physically unmotivated, and (iii) the identification of the correct anisotropic tree-level formula. Items (ii) and (iii) occurred in the same debugging episode but represent distinct cognitive acts: rejecting a wrong answer versus finding the right one.

4. Three Principles

The case study suggests three principles for supervising AI agents on scientific software, each grounded in a specific failure mode we observed.

P1: Oracle testing verifies what, not why. An oracle test compares numerical output at fixed input parameters. It answers the question "does the code produce the right numbers here?" but not "does the code produce the right numbers *for the right reasons*?" The fudge factor $\alpha = 0.27$ passed every oracle test (nine spectra, hundreds of k-modes, relative errors below 1%) because it was calibrated to the same fiducial cosmology used for testing. The oracle provides no signal that the underlying model is wrong when the calibration point happens to coincide with the test point.

A subtler instance appeared during the upstream Boltzmann

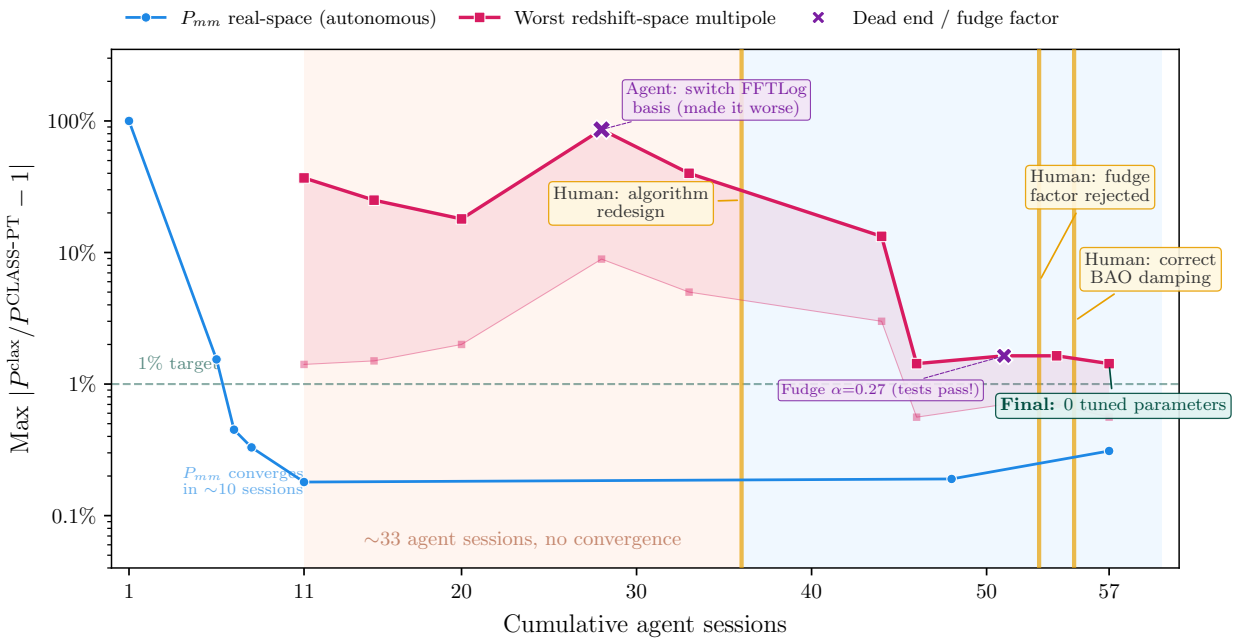

Figure 2. Accuracy convergence over 57 agent sessions. Blue: real-space matter power spectrum (autonomous, converges in ∼10 sessions). Red: worst redshift-space multipole (stuck at 8–86% for ∼33 sessions). Vertical gold lines mark human interventions. The agent's error plateaued because the code architecture was structurally incompatible with anisotropic BAO damping—no coefficient adjustment within that architecture could make all multipoles pass simultaneously. After the physicist directed a Gauss–Legendre quadrature redesign, errors dropped to the 1–2% range. A subsequent fudge factor ($\alpha = 0.27$, purple cross) passed all tests but was rejected by the physicist as physically unmotivated. The correct formula required zero tuned parameters.

solver development: the agent misdiagnosed a proton-to-hydrogen mass ratio typo ($m_p/m_H = 1.0$ vs. the correct 0.9994) as "recombination solver bias" and committed compensating changes that passed all tests for months before the one-character error was found.

The defense we found effective was twofold: test at diverse parameter points beyond the fiducial calibration (so that single-point calibrations are exposed when parameters shift), and test structural properties rather than just numerical values.

P2: Shared memory prevents re-exploration but not architectural loops. The CHANGELOG protocol successfully prevented the agent from re-exploring solved bugs across sessions. When the DST k-grid issue (logarithmic vs. linear spacing) was resolved on day 3, no subsequent session revisited it—the CHANGELOG entry served as shared memory. However, the same protocol did not prevent 33 sessions of coherent but futile work within the wrong architecture. Sessions during the RSD crisis logged their attempts faithfully, showing no contradiction or repetition—only incremental, methodical exploration within a space that contained no solution.

The shared memory prevented *literal* re-exploration (trying the same coefficient twice) but could not surface *structural* stagnation (trying different coefficients within the same doomed architecture). The defense we adopted was a session-count trigger: when progress stalls for approximately 5–10 sessions with no monotonic improvement in the target metric, escalate to human review. This heuristic would have caught the RSD crisis at session 30 rather than session 56.

P3: The irreducible human role is architectural and physical judgment. The agent excelled at tasks with clear specifications and verifiable outputs: transcribing equations from a reference source, comparing intermediate numerical values to localize discrepancies, adjusting coefficients to match a known target, implementing a well-specified algorithm. These tasks, which constituted approximately 80% of the project by session count, were performed autonomously at high quality.

The agent failed at two specific tasks. First, recognizing when a code architecture is fundamentally incompatible with the target physics rather than merely mis-parameterized. This requires understanding what the physics *should look like*—not just what numbers it should produce, but what

structural properties must hold (isotropy vs. anisotropy, polynomial vs. exponential dependence, which variables are coupled). Second, detecting physically unmotivated patches that happen to improve numerical agreement. This requires knowing what the theory *allows*—that a scalar correction α must derive from some identifiable physical mechanism, and that its absence from the reference implementation is evidence against its validity rather than evidence of a novel improvement.

Both reduce to a single underlying skill that we term *explanatory agency*: evaluating explanations, not just predictions. The agent evaluates whether the output matches the target. The physicist evaluates whether the mechanism producing that output is physically coherent. When these two criteria diverge—when a wrong mechanism produces right numbers—only the physicist can detect the error.

5. Implications and Limitations

The division of labor that emerged (classified retrospectively by the supervising physicist from the CHANGELOG and git history without independent coding, with each session defined as a single Claude Code invocation typically lasting 15–45 minutes) was: approximately 80% of sessions were autonomous oracle-following (transcription, debugging, implementation), approximately 10% were engineering tasks where the agent worked independently given a well-specified architecture, and approximately 10% required irreplaceable human judgment (architectural decisions and fudge-factor rejection). The binding constraint on code quality was the supervision protocol, not the model's raw capability.

This suggests that improving AI-assisted scientific software requires better supervision protocols at least as much as better models. Specifically, it requires protocols that distinguish predictive adequacy from explanatory correctness. The "no fudge factors" rule is a crude but effective instance that forces mechanistic explanations rather than calibrated patches.

We acknowledge significant limitations. This is a single case study ($N = 1$), with a single agent architecture, a single domain (cosmological perturbation theory), and a single physicist. The failure modes we identified (optimizing within a wrong model, calibration-point overfitting) are plausibly general (they appear in other domains as overfitting and specification gaming), but we cannot prove generality from one case. The project also had a high-quality oracle available (CLASS-PT is an established, well-tested reference code). Many scientific software projects lack such oracles, and our methodology would need adaptation for settings where ground truth is uncertain.

Although our evidence is observational, the specificity of the failure modes permits qualitative counterfactual analysis that grounds the three principles in testable claims. Had the "no fudge factors" rule not been in place, the $\alpha = 0.27$ correction would have shipped—it passed all nine spectra, was committed by the agent as the solution, and would have been invisible to any downstream consumer of the code. Had multi-cosmology testing been enforced from day one rather than added after the fudge factor was discovered, the calibration would have been caught sooner: $\alpha = 0.27$ is fit to fiducial Planck 2018 parameters and would produce visibly wrong BAO amplitudes at $\omega_b \pm 20\%$. Had the session-count escalation trigger been formalized at 10 sessions rather than applied informally around session 50, the 33-session architectural loop would have been interrupted approximately 23 sessions earlier, saving roughly 40% of the total project time.

Looking forward, two concrete remediation paths could narrow the gap between autonomous and human-required bugs without requiring qualitative advances in reasoning. First, retrieval-augmented reasoning over the full reference codebase, including code paths the agent did not initially consult, could surface relevant physics without human guidance. In our case, the correct formula existed in a CLASS-PT branch the agent never consulted (Section 3.2). A retrieval system indexing all branches would have surfaced it when the isotropic architecture failed to converge. Second, explicit "physics audit" prompting (systematically asking the agent whether every tuned parameter corresponds to a physical quantity in the reference theory) would operationalize the diagnostic question that caught the fudge factor. The physicist's intervention reduced to a single query ("does $\alpha = 0.27$ appear anywhere in CLASS-PT?"), which the agent answered correctly once asked but could not generate unprompted. Embedding such queries as mandatory checkpoints after any parameter introduction would convert an ad hoc human insight into a reproducible protocol step. Both paths are concrete, testable, and complementary to scaling: they address the evaluation gap rather than the reasoning gap.

5.1. Credit, Attribution, and Governance

The division of labor raises an unavoidable question about credit. The agent performed approximately 80% of sessions (implementation, debugging, transcription) while the physicist performed approximately 10% but supplied 100% of the decisive architectural and physical judgments. We argue that contribution weight should reflect *irreplaceability*, not volume: the physicist's three interventions (the Gauss–Legendre redesign, the fudge-factor rejection, and identification of the correct anisotropic formula) were load-bearing in a way that the agent's 10 autonomous bug fixes were not. Any sufficiently capable coding agent could have resolved convention errors given an oracle test suite. No agent in this

study could have diagnosed the architectural mismatch or rejected a passing-but-unphysical solution.

The analogy to graduate advising is instructive—and more uncomfortable for the "tool, not co-author" framing than it may first appear. The physicist's supervision was indistinguishable from advising a student: pedagogical notes on the physics, guidance on debugging strategy rather than direct code review, and direction on which tests to run. A graduate student who performed the same work would be lead author. What disqualifies the agent is its lack of explanatory agency: not merely producing correct output, but understanding *why* it is correct and defending that understanding under scrutiny. A student who discovered $\alpha = 0.27$ would develop physical discomfort with an unphysical parameter and eventually ask the advisor's question unprompted. The agent could not, because it evaluates predictions rather than explanations. More practically, a co-author takes intellectual responsibility: defending the work at conferences, responding to referees, carrying lessons forward. Explanatory agency is developmental: students acquire it through years of domain immersion, and future agents may develop it through advances beyond scaling. Until they do, authorship remains a human prerogative even when the work product suggests otherwise.

The fudge factor ($\alpha = 0.27$) sharpens the liability question. Had this parameter shipped in a fully autonomous pipeline (analogous to Denario, Villaescusa-Navarro et al. 2025), it would have produced subtly wrong predictions at non-fiducial cosmologies, and the oracle test suite would not have caught it. Who bears responsibility: the model developer, the scientist who deployed the pipeline, or the institution that sanctioned its use? Current frameworks provide no clear answer, precisely because the error is not a software bug but a *scientific* mistake that requires domain expertise to detect.

We propose that AI-assisted scientific software should ship with a **supervision log** as provenance documentation, analogous to a lab notebook. The log records which decisions were human-directed (the architectural redesign) versus agent-proposed (the fudge factor), enabling post-hoc audit of the chain of responsibility. We will release ours alongside the code and CHANGELOG. Finally, we note a scalability concern: the supervision practices P1–P3 worked for one physicist and one agent because the physicist had sufficient domain expertise to enforce the "no fudge factors" rule. When thousands of laboratories deploy coding agents on scientific software, these practices must become institutional (embedded in review processes, CI pipelines, and reproducibility standards) rather than relying on the vigilance of individual supervisors.

6. Related Work

Carlini (2026) reported on building a C compiler using 16 parallel Claude agent sessions over approximately 2,000 sessions, producing a 100,000-line Rust implementation that compiles the Linux kernel. Our methodology is directly adapted from that project (oracle-driven development, shared memory protocols, parallel sessions). The key difference is that the C compiler domain requires syntactic and semantic correctness (fully captured by oracle tests, GCC output) but not *physical* correctness. Indeed, the compiler setting is precisely the case where oracle testing *is* sufficient—the absence of a gap between predictive and explanatory correctness is what made fully autonomous development feasible there. Our case study extends the Carlini methodology to a domain where this gap exists and is load-bearing.

Villaescusa-Navarro et al. (2025) describe Denario, a multi-agent system that autonomously generates scientific analyses and manuscripts in astrophysics. Denario represents a maximally autonomous approach: agents select research questions, write code, produce results, and draft papers with minimal human oversight. Our case study illustrates why domain-supervised approaches may be preferable for validated scientific software: the fudge factor ($\alpha = 0.27$) that our agent committed, which passed all automated tests, would have been published without question in a fully autonomous pipeline. The supervision protocol catches errors that autonomy cannot.

The fudge factor failure mode connects to a broader phenomenon in AI alignment: specification gaming, where an agent optimizes a proxy metric until it ceases to measure the intended objective (Krakovna et al., 2020). The $\alpha = 0.27$ correction is a domain-specific instance—the agent optimized test-suite error (the proxy) at the expense of physical correctness (the true objective). Our "no fudge factors" rule operationalizes the general defense: constrain optimization to exclude solutions that game the metric.

SymBoltz.jl (Sletmoen, 2026) implements a differentiable linear Boltzmann solver. Our contribution is orthogonal: we report on *how* such code is developed when the implementer is an AI agent under human supervision.

Not all AI-for-science applications require this level of domain supervision. In protein structure prediction (Jumper et al., 2021) and materials discovery (Merchant et al., 2023), the oracle (experimental structure data or DFT calculations) captures the full correctness criterion. There is no gap between "right numbers" and "right physics" because the target *is* the numbers. Our case study applies specifically to domains where correctness requires agreement with physical theory, and where the theory admits multiple numerically adequate but physically distinct implementations.

7. Conclusion

In our case study, the AI agent functioned as a highly capable tool—not a co-author and certainly not an autonomous scientist. It could implement, debug, and validate scientific software at speed and scale impractical for a solo physicist, but it lacked the explanatory agency to judge whether its solutions were physically meaningful—whether they produced right numbers for the right reasons. The supervision protocol (oracle testing, shared memory, the no-fudge-factor rule, and the session-count escalation trigger) was the mechanism that transformed raw agent output into trustworthy scientific code. Improving this protocol, rather than solely improving model capability, is the more direct path to reliable AI-assisted scientific software development. The full supervision log, CHANGELOG, and commit history will be made publicly available.

References

Baumann, D., Nicolis, A., Senatore, L., and Zaldarriaga, M. Cosmological Non-Linearities as an Effective Fluid. *JCAP*, 07:051, 2012. doi: 10.1088/1475-7516/2012/07/051.

Carlini, N. Coding agent builds a C compiler. Anthropic Technical Report, https://anthropic.com/research/coding-agent-c-compiler, 2026.

Carroll, S. M., Leichenauer, S., and Pollack, J. Consistent effective theory of long-wavelength cosmological perturbations. *Phys. Rev. D*, 90(2):023518, 2014. doi: 10.1103/PhysRevD.90.023518.

Chudaykin, A., Ivanov, M. M., Philcox, O. H. E., and Simonović, M. Nonlinear perturbation theory extension of the boltzmann code CLASS. *Phys. Rev. D*, 103:023507, 2021. doi: 10.1103/PhysRevD.103.023507. https://github.com/Michalychforever/CLASS-PT.

D'Amico, G. et al. The cosmological analysis of the SDSS/BOSS data from the effective field theory of large-scale structure. *JCAP*, 2020(05):005, 2020.

Fang, X., Blazek, J. A., McEwen, J. E., and Hirata, C. M. FAST-PT II: an algorithm to calculate convolution integrals of general tensor quantities in cosmological perturbation theory. *JCAP*, 2017(02):030, 2017.

Jumper, J. et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596:583–589, 2021. doi: 10.1038/s41586-021-03819-2.

Krakovna, V., Uesato, J., Mikulik, V., Rahtz, M., Everitt, T., Kumar, R., Koop, Z., Lefrancq, A., and Legg, S. Specification gaming: the flip side of AI ingenuity. DeepMind Blog, 2020.

Merchant, A. et al. Scaling deep learning for materials discovery. *Nature*, 624:80–85, 2023. doi: 10.1038/s41586-023-06735-9.

Perko, A., Senatore, L., Jennings, E., and Wechsler, R. H. Biased tracers in redshift space in the EFT of large-scale structure. *arXiv preprint arXiv:1602.00674*, 2016.

Senatore, L. and Zaldarriaga, M. The IR-resummed effective field theory of large scale structures. *JCAP*, 2015(02):013, 2015. doi: 10.1088/1475-7516/2015/02/013.

Simonović, M., Baldauf, T., Zaldarriaga, M., Carrasco, J. J., and Kollmeier, J. A. Cosmological perturbation theory using the FFTLog: formalism and connection to QFT loop integrals. *JCAP*, 04:030, 2018. doi: 10.1088/1475-7516/2018/04/030.

Sletmoen, H. SymBoltz.jl: A symbolic-numeric, approximation-free, and differentiable linear Einstein–Boltzmann solver. *Astronomy & Astrophysics*, 707:A128, 2026. doi: 10.1051/0004-6361/202557450.

Villaescusa-Navarro, F. et al. The Denario project: Deep knowledge AI agents for scientific discovery. *arXiv preprint arXiv:2510.26887*, 2025.

Vlah, Z., White, M., and Aviles, A. A Vlasov-Poisson approach to the large-scale structure, with applications to the EFT. *JCAP*, 2015(09):014, 2015. doi: 10.1088/1475-7516/2015/09/014.